

Reguläre Ausdrücke

grep und **sed** entfalten ihre Mächtigkeit erst mit dem Verwenden von Regulären Ausdrücken. Diese können sehr komplex sein und nicht einfach zu lesen. In diesem Dokument werden ein paar einfache Möglichkeiten vorgestellt.

Wenn wir im Dateisystem nach einer Datei suchen, deren Namen wir nicht genau kennen oder nach mehreren Dateien, dann arbeiten wir mit Platzhaltern. Wir schreiben zum Beispiel `ls -l *.png` und es werden uns alle PNG-Dateien angezeigt.

Reguläre Ausdrücke arbeiten nach dem gleichen Prinzip. Es gibt Platzhalter, die bei der Suche aufgelöst werden. Nur gibt es sehr viel mehr verschiedene Platzhalter. Der resultierende Suchausdruck ist dann häufig nicht mehr intuitiv lesbar.

Hinweise:

- Für alle Beispiele wird **nicht** unterschieden zwischen Gross- und Kleinschreibung.
- Die Beispiele in diesem Dokument wurden mit der Webseite <https://regexr.com/> erstellt. Ich empfehle Ihnen, ebenfalls mit dieser Seite zu arbeiten oder mit <https://regex101.com/>. Oben im Fenster ist jeweils der Reguläre Ausdruck zu sehen. Unten ist eine Liste von Ausdrücken. Diese sind farbig markiert, wenn es eine Übereinstimmung mit dem Ausdruck gibt.

Expression
/i/gi
Text
Meier↵
Meyer↵
Maier↵
Mayer↵
Obermeier↵
Meierhofer↵

Einfache Suche

Wir können auch mit Regulären Ausdrücken nach ganz einfachen Worten suchen. So ergibt der Suchausdruck `meier` die folgenden Übereinstimmungen:

Expression
<code>/meier/gi</code>
Text
Meier ↗
Meyer ↗
Maier ↗
Mayer ↗
Obermeier ↗
Meierhofer ↗

Suche mit Zeichenklassen (character classes)

Wenn wir unsicher sind, ob sich die gesuchte Person Meier oder Meyer schreibt, können wir eine character class bilden. Dies geschieht durch eckige Klammern `[ ]`. In dieser können wir Zeichen auflisten, die gültig sind.

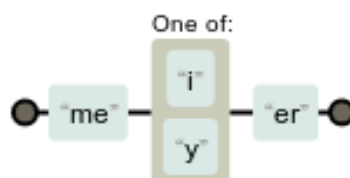
Achtung: eine eckige Klammer steht nur für 1 Zeichen im Text

Der Ausdruck `me[iy]er` findet dann folgendes:

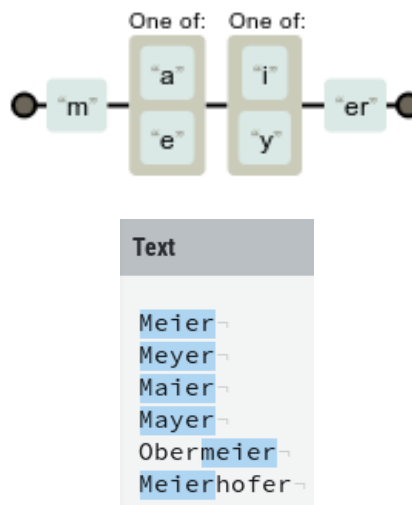
Text
Meier ↗
Meyer ↗
Maier ↗
Mayer ↗
Obermeier ↗
Meierhofer ↗

Der Ausdruck `me[iy]er` ist noch einfach zu lesen. Trotzdem wird hier begonnen, die Ausdrücke jeweils noch zu visualisieren. Dies wurde mit der Webseite <https://regexper.com/> gemacht.

Bei diesen Diagrammen, wird einfach von links nach rechts der schwarzen Linie gefolgt. Bei Verzweigungen kann man wählen, welcher Linie man folgen möchte.



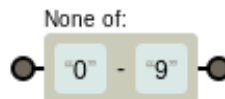
Wenn wir jetzt auch noch Maier und Mayer in die Suche einschliessen möchten, dann formulieren wir `m[ae] [iy]er`



Statt nur einzelnen Zeichen können wir in Zeichenklassen Bereiche angeben, wenn die gesuchten Zeichen beieinander liegen. Das geschieht mit dem Bindestrich. Ein paar Beispiele:

- `[a-z]`
- `[0-9]`
- `[a-zA-Z]`
- `[0-9a-zA-Z]`

Der Inhalt einer Zeichenklasse lässt sich auch negieren. Wenn wir schreiben `[^0-9]` dann ist jedes Zeichen erlaubt, nur nicht eine Ziffer.



Um die Arbeit mit Zeichenklassen zu vereinfachen, gibt es vordefinierte Zeichenklassen. Hier sind die gebräuchlichsten davon:

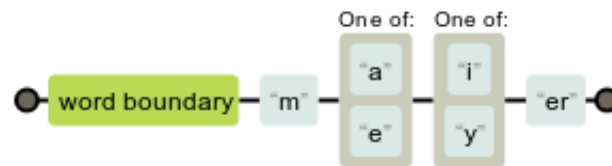
- `\b` (boundary) steht für eine Wortgrenze
- `\s` (space) steht für Whitespace (also Leerzeichen, Tabulator, Zeilenende, etc.)

Ein Spezialfall ist der Punkt (`.`). Der steht für ein beliebiges Zeichen. Wenn Sie wirklich nach einem Punkt suchen, müssen Sie nach `\.` suchen.

Zur Erinnerung: eine Zeichenklasse (also eine eckige Klammer `[ ]`) steht nur für ein einzelnes Zeichen. Wenn nach einer Wiederholung solcher Zeichen gesucht werden soll, benötigen Sie Wiederholungszeichen. Dazu folgt weiter unten im Dokument mehr.

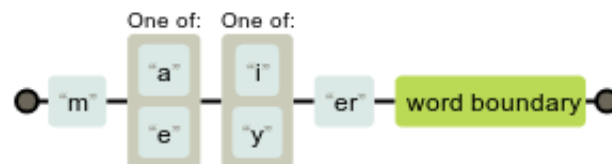
Suche mit der Wortgrenze

Es fällt auf, dass die Namen Obermeier und Meierhofer ebenfalls erscheinen. Wenn wir das nicht möchten, können wir zum Beispiel verlangen, dass der gesuchte Ausdruck am Wortanfang stehen muss. Dazu dient der Platzhalter `\b`, also: `\bm[ae][iy]er`



Text
Meier
Meyer
Maier
Mayer
Obermeier
Meierhofer

Wir können natürlich auch das Wortende verlangen: `m[ae][iy]er\b`

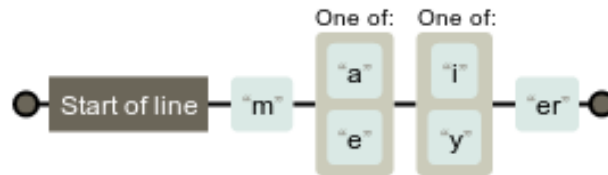


Text
Meier
Meyer
Maier
Mayer
Obermeier
Meierhofer

Suche mit Zeilenanfang und Zeilenende

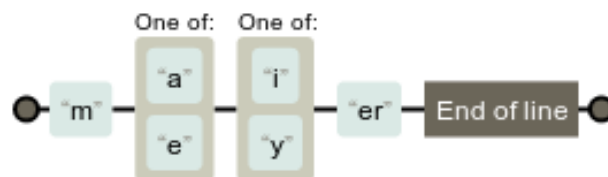
Wir können auch den Zeilenanfang oder das Zeilenende verlangen. Das sieht dann so aus:

`^m[ae][iy]er`



Text
Peter•Meier↵
Meyer•Peter↵
Peter•Maier-Müller↵

`m[ae][iy]er$`



Text
Peter•Meier↵
Meyer•Peter↵
Peter•Maier-Müller↵

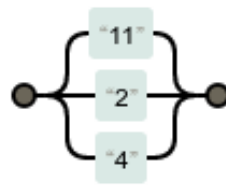
Alternativen und Gruppierung

Bei der Suche nach Ausdrücken können wir auch Alternativen angeben. Nehmen wir an, wir suchen nach den Zahlen 11, 12 und 14. Dann können wir mit dem senkrechten Strich | Suchalternativen angeben. Aber meistens muss dazu eine Gruppierung verwendet werden.

Hier zuerst ein falsches Beispiel. (Dieses Problem könnte man auch mit einer character class lösen, aber hier geht es darum, Alternativen vorzustellen.)

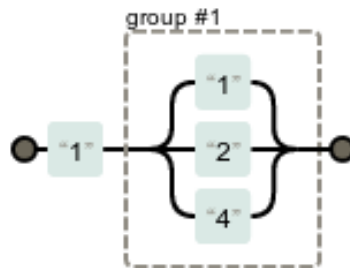
Wenn ich verlange, dass nach der ersten 1 entweder eine 1, eine 2 oder eine 4 kommen muss, könnte ich das so formulieren: `11|2|4`

Aber das führt nicht zum gewünschten Ergebnis. Das wird nämlich so interpretiert:



Korrekt muss die Aufgabenstellung so formuliert werden: `1(1|2|4)`

Damit werden die gewünschten Zahlen gefunden:



Text
10 ↗
11 ↗
12 ↗
13 ↗
14 ↗
20 ↗
21 ↗

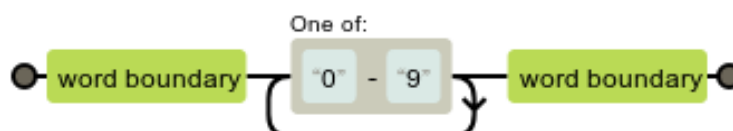
Wiederholungen

Wir können auch verlangen, dass sich gewisse Elemente wiederholen. Das können wir mit vordefinierten Platzhaltern machen oder selbst die Grenzen angeben.

Die vordefinierten Platzhalter sind:

- `*` - 0 bis beliebig häufig
- `+` - 1 bis beliebig häufig
- `?` - 0 bis 1 mal

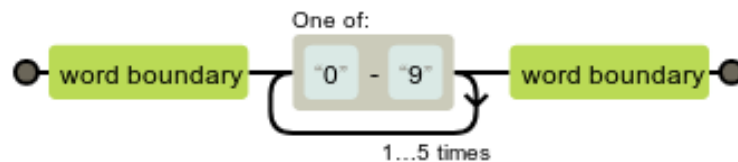
Wenn ich zum Beispiel eine Zahl einlesen möchte, die beliebig lang sein darf, dann kann ich das so ausdrücken: `\b[0-9]+\b`



Text

```
1
232342
31342123
```

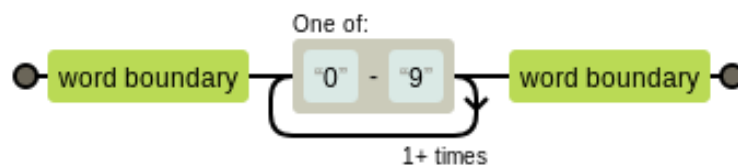
Die Anzahl Wiederholungen kann auch selbst gewählt werden. Dazu verwendet man die geschweiften Klammern. Darin gibt man die minimale und die maximale Anzahl an. `\b[0-9]{2,6}\b`



Text

```
1
232342
31342123
```

Gibt es keine obere Grenze, folgt nach dem Komma gleich die schliessende Klammer. `\b[0-9]{2,}\b`



Beispiel

Versuchen wir nun ein paar Dinge, die besprochen wurden, in einem Beispiel anzuwenden. Wir suchen einen Ausdruck, um in einem Datum einen gültigen Tag zu erkennen. Achtung, es geht nur darum, einen gültigen Wert von 1 - 31 zu erkennen.

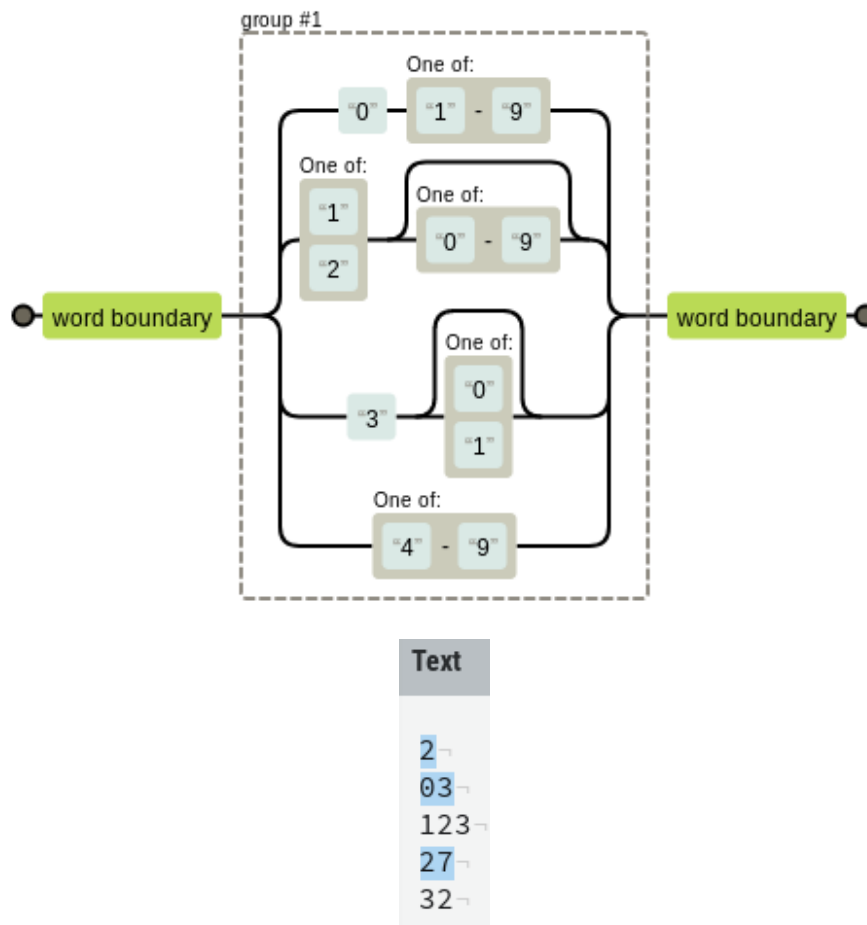
Wenn wir analysieren, kommen wir auf folgende Regeln:

- die Zahl darf mit einer 1 beginnen, auf die höchstens eine andere Ziffer folgt
- die Zahl darf mit einer 2 beginnen, auf die höchstens eine andere Ziffer folgt
- die Zahl darf mit einer 3 beginnen, auf die entweder keine Ziffern oder eine 0 oder 1 folgen
- die Zahl darf mit einer 4 - 9 beginnen, es darf dann aber keine andere Ziffer folgen

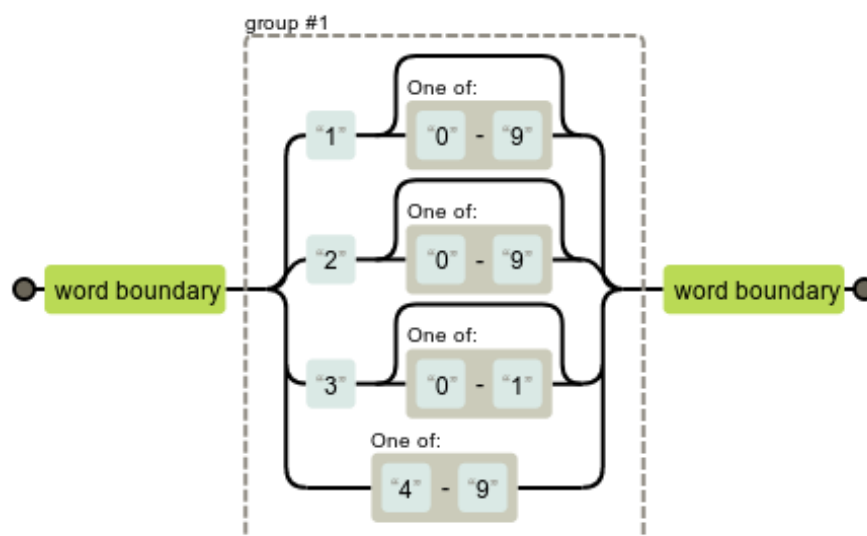
Es gibt mehr als eine Lösung, das zu formulieren. Hier zwei Beispiele:

1. `\b(0[1-9] | [12] [0-9]? | 3[01]? | [4-9])\b` (diese Variante akzeptiert führende Nullen)
2. `\b(1[0-9]? | 2[0-9]? | 3[0-1]? | [4-9])\b`

Variante 1:



Variante 2:



Text
123 ↵
23 ↵
31 ↵
32 ↵

Wenn Sie Übungen für Reguläre Ausdrücke suchen, können Sie sich auf der Webseite <https://alf.nu/RegexGolf> versuchen.